

I N S T R U C T O R - L E D   P R O G R A M   •   4 8   H O U R S   •   1 2   H A L F - D A Y   M O D U L E S

D A Y   1   /   M O D U L E   1

# Program Foundation & the Agentic Engineering Model

---

*How AI generates options, humans review, and we build production-minded airline data systems on AWS.*

Prepared for **Accelya Kale** • Delivered by **Jai Singh**

# About the trainer

## Jai Singh

*Independent technology trainer, consultant, and solutions provider*

**20+ years** across military, technology leadership, fractional CTO work, and corporate training.

- **Delivery footprint:**
  - Trained engineers across corporate clients.
- **Areas of expertise:**
  - AI/ML, agentic systems, hyperscaler cloud (AWS, GCP, Azure), DevOps, full-stack engineering, air-gapped AI infrastructure.

## What to expect from this program

### Hands-on first

Roughly 55–60% of every 4-hour module is labs on a trainer-managed AWS sandbox.

### Airline vocabulary

PNR, fare class, OD, settlement, remittance — woven into every prompt, schema, and lab.

### Human-in-the-Loop

No AI-generated code merges without a human review pass against the course checklist.

### Approved lab stack

Windsurf IDE with Cascade, GitLab, AWS Bedrock, and AWS services. Stack approved for the engagement.

### Production-minded

Every artefact is modular, observable, cost-aware, and reviewable from Day 1.

# Day 1 agenda

A standard half-day module: concepts, guided design, hands-on labs, then a short review wrap.

1

30–35 min

## Concept briefing

What agentic development really means in 2026. HITL architecture. Workshop operating model. Batch vs streaming vs inference. Airline vocabulary. AI/data safety.

2

35–40 min

## Guided design

Tooling stack walkthrough. The HITL flow: prompt → options → review → build → validate. Course review checklist as a real artefact. Repo structure tour.

3

2 – 2¼ hrs

## Hands-on labs

(1) Verify Windsurf + Cascade + GitLab + AWS Bedrock access. (2) One-line airline BR → technical prompt. (3) Generate two architecture options and run a HITL review.

4

20–30 min

## Review, Q&A, wrap

Module 1 takeaways. Common HITL mistakes. Open discussion. End-of-module cleanup. Day 2 preview.

*Audience: mixed across Foundation (≤1 yr), Working (3+ yrs), and DevOps/Senior (3.5+ yrs) bands. Pair-programming during labs.*

# Day 1 learning outcomes

## DEFINE

Articulate what 'agentic development' means in 2026 — and how it differs from autocomplete-only coding assistants.

## DECIDE

Choose between batch, streaming, and real-time inference for an airline workload.

## REVIEW

Score an AI-generated architecture against the course HITL checklist and pick a winner.

## APPLY

Run the HITL flow end-to-end: prompt → options → review → build → validate.

## BUILD

Verify your Windsurf + Cascade + GitLab + AWS Bedrock + AWS sandbox is wired correctly.

## SPEAK

Use airline vocabulary (PNR, fare class, OD, settlement) accurately in prompts and artefacts.

*These outcomes are reinforced through a written individual readiness check in Module 12.*

# 01

PART 1 • ~ 30 – 35 MINUTES

## Concept briefing

Seven concepts that frame the next 11 modules: what agentic development actually is, why HITL is non-negotiable, the workshop operating model, the batch/streaming/inference split, the airline vocabulary, and the data-safety guardrails.

# What “agentic development” actually means

*AI agents that plan, generate, modify, test, and explain software — across multiple stages of the SDLC, working alongside human developers with a degree of autonomy.*

## FORRESTER

“Agentic Software Development (ASD) is a distinct category — with its own practices, tools, and implications for the workforce.”

*Forrester, Mar 2026*

## PRINCIPLE

Principle of agentic engineering — the human-driven discipline. AI is a tool, not the force building the codebase end-to-end.

*Trainer framing*

## STACK OVERFLOW

84% of developers use or intend to use AI-assisted programming. 46% remain skeptical of accuracy.

*Stack Overflow Developer Survey 2025*

**In this program: agentic = the AI proposes, the human reviews, the code ships.**

# Three modes of AI-assisted engineering

Each mode requires a different review discipline. This program lives mostly in the middle.

## Autocomplete assist

*Tab completions, chat-based snippets*

EXAMPLE PROMPT

“Write a Python lambda handler that parses this JSON.”

|            |                 |
|------------|-----------------|
| Autonomy   | Review depth    |
| <b>LOW</b> |                 |
|            | Glance & accept |

## Agentic development

*Cascade in Windsurf · multi-file edits, plans, test runs*

EXAMPLE PROMPT

“Scaffold the ticket-issue processor with validator, handler, tests.”

|               |                              |
|---------------|------------------------------|
| Autonomy      | Review depth                 |
| <b>MEDIUM</b> |                              |
|               | HITL checklist (this course) |

## Autonomous coding agent

*Devin, Claude Code, autonomous MR-opening agents*

EXAMPLE PROMPT

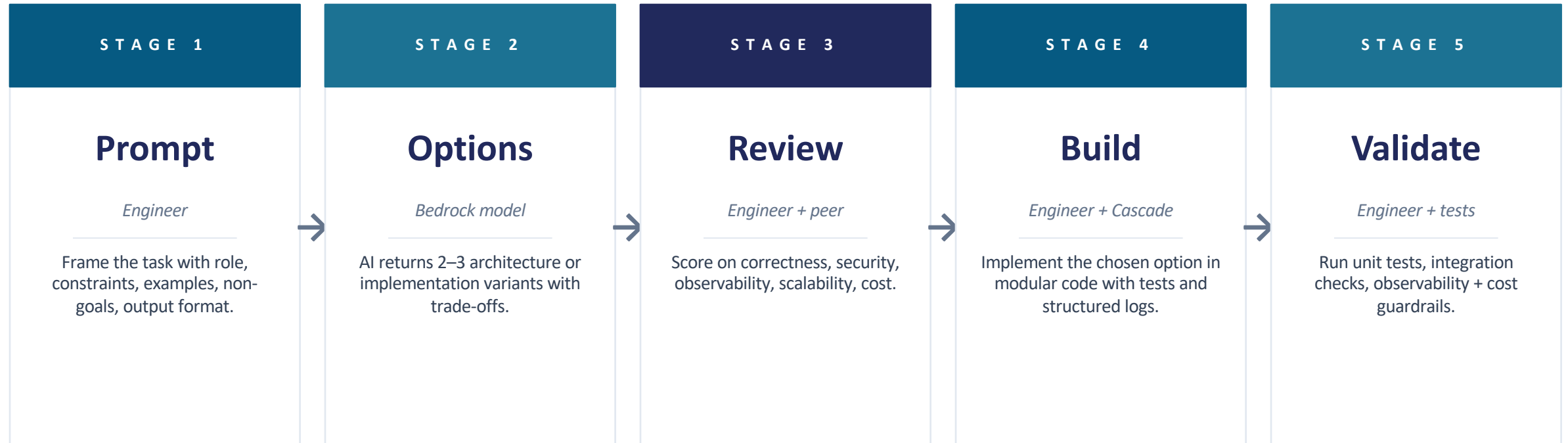
Assigned a GitLab issue, opens an MR overnight.

|             |                                      |
|-------------|--------------------------------------|
| Autonomy    | Review depth                         |
| <b>HIGH</b> |                                      |
|             | Full code review + integration tests |

*This program: middle column. We use Cascade for multi-file work and run every output through HITL.*

# The five-stage HITL flow

Every artefact in this course — BRDs, architectures, code, tests — moves through these five stages.



**Non-negotiable:** no AI-generated artefact moves to the next stage without an explicit human pass against the course HITL checklist.

# Five review dimensions for every AI output

If you can't tick all five with confidence, the artefact is not ready to ship.

01

## Correctness

Does it actually do what the prompt asked? Edge cases handled? Real APIs, not hallucinated ones?

02

## Security

IAM least-privilege? Secrets out of code? PII handling? Injection-safe? No public buckets?

03

## Observability

Structured JSON logs? Correlation IDs? Metrics emitted? Errors classified, not silently swallowed?

04

## Scalability

Will it survive 10× the data? Idempotent? Backpressure? Stateless where possible?

05

## Cost

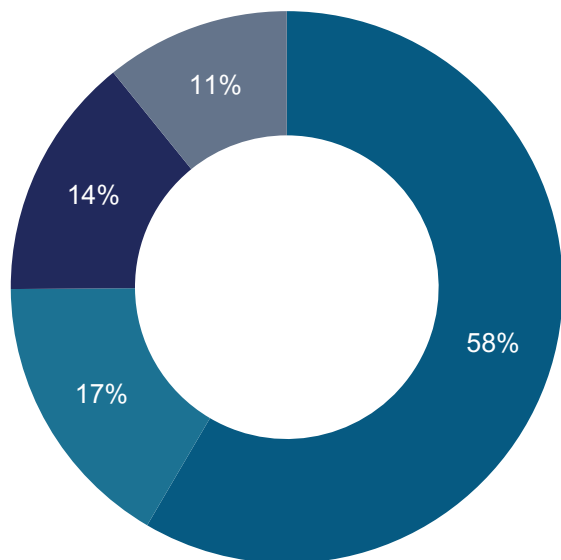
Memory · timeout · batch size right-sized? Athena scan reduced via partitioning? Bedrock token usage bounded?

**Gating rule (programme-wide; locked in Day 3): each dimension scored 0-3 (0=missing · 1=weak · 2=acceptable · 3=strong). PASS = total  $\geq 12$  of 15 AND every dimension  $\geq 2$ . All unmet dimensions documented in HITL\_REVIEW.md.**

**Role split:** Senior owns Security and IAM. Working-band drives Correctness and Scalability. Foundation-band asks the questions.

# Workshop operating model

All 12 modules use the same rhythm. Predictable structure means cognitive energy stays on the work.



■ Hands-on lab (~2.25h) ■ Guided design (~38m)  
 ■ Concept (~33m) ■ Review & wrap (~25m)

**30–35 min**

## Concept briefing

Topic, reference architecture, airline tie-in. Your eyes follow slides; not a coding hour.

**35–40 min**

## Guided design

AI generates options. Group reviews. Winner selected against the HITL checklist.

**2–2½ hrs**

## Hands-on lab

Implement on the AWS sandbox in Windsurf. Cascade pairs with you. Tests written here.

**20–30 min**

## Review & wrap

Q&A. Peer review pass. Cost / IAM / observability checklist. End-of-module cleanup.

*Modules 1–3 (Foundation) front-load concepts. From Module 4, lab time grows.*

# Lab deliverable & review standards

## A LAB DELIVERABLE LOOKS LIKE ...

- **GitLab branch** named accelya-m1-<your-name> with the lab work committed.
- **Modular folder tree** (handler / service / validator / config / tests) — not one fat file.
- **Structured JSON logs** with correlation IDs and an airline-aware payload field set.
- **Unit + functional tests** covering happy path, schema failure, and missing-metadata cases.
- **Prompt log** saved in /prompts/ — the prompt that produced your scaffold, plus its review notes.
- **Cleanup script** that tears the resources down at end of module.

## A GOOD REVIEW LOOKS LIKE ...

- **Reviewer reads the prompt first** — you can't judge an output without knowing what was asked.
- **Ticks all five HITL dimensions** (correctness, security, observability, scalability, cost) — not just “does it run.”
- **Names hallucinations explicitly** — fake API names, missing error paths, confidently wrong limits.
- **Asks for tests by category** — not “add tests” but “add a test for malformed PNR.”
- **Suggests, doesn't rewrite** — the author iterates with the model. Reviewers don't do silent rewrites.

# Batch · Streaming · Real-time inference

Modules 4–10 cover all three. Picking the right pattern matters more than picking the perfect framework.

BATCH

Files arrive on a schedule

AIRLINE EXAMPLE

Daily ticket-issue files → validate → curate for settlement reporting.

|          |                                             |
|----------|---------------------------------------------|
| Latency  | Minutes – hours                             |
| Cost     | Low                                         |
| Services | S3 · Lambda · Glue · Athena                 |
| Best for | Reconciliation · reporting · settlement ETL |

COVERED IN Modules 4 – 6

STREAMING

Events arrive continuously

AIRLINE EXAMPLE

Booking, cancellation, refund events flowing through the day.

|          |                                                    |
|----------|----------------------------------------------------|
| Latency  | Seconds                                            |
| Cost     | Medium                                             |
| Services | Kinesis · Lambda · on-failure destination          |
| Best for | Live dashboards · alerts · event-driven enrichment |

COVERED IN Modules 7 – 8

REAL-TIME INFERENCE

Decisions in the loop

AIRLINE EXAMPLE

Refund recommendation or anomaly flag on each cancellation event.

|          |                                                         |
|----------|---------------------------------------------------------|
| Latency  | Milliseconds                                            |
| Cost     | Higher (per call)                                       |
| Services | SageMaker Serverless · Bedrock                          |
| Best for | Per-event scoring · recommendations · anomaly detection |

COVERED IN Module 9

# Five airline use cases — which pattern wins?

| Airline use case                            | Batch | Streaming | Real-time inference | Why                                              |
|---------------------------------------------|-------|-----------|---------------------|--------------------------------------------------|
| Daily ticket-issue → settlement reporting   | ✓     | —         | —                   | Files land on schedule; latency does not matter. |
| Live booking dashboard for ops              | —     | ✓         | —                   | Sub-second freshness; high event volume.         |
| Cancellation → refund recommendation        | —     | ✓         | ✓                   | Streamed event triggers per-event model call.    |
| Quarterly remittance reconciliation         | ✓     | —         | —                   | Cost-sensitive; aggregate over a window.         |
| Anomaly detection on settlement adjustments | —     | ✓         | ✓                   | Per-event scoring with stream context.           |

Many real-world flows are hybrids: a batch reconciliation layer underneath a streaming hot path. We will build both in this course.

Legend: ✓ fits well — not appropriate

# Airline vocabulary that runs through every module

These terms appear in every prompt, payload, schema, log line, and review across the next 11 modules.

|                                                                           |                                                              |                                                                        |                                                                   |
|---------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>PNR</b><br>Passenger Name Record — booking identifier.                 | <b>Airline code</b><br>2-letter IATA code: AI, BA, EK, AA... | <b>OD</b><br>Origin–Destination pair: BOM→DEL.                         | <b>Fare class</b><br>Booking class controlling rules and price.   |
| <b>Fare break</b><br>Point where a fare splits into stop/connecting legs. | <b>Offer</b><br>Priced product the GDS returns for a search. | <b>GDS</b><br>Global Distribution System (e.g. Amadeus).               | <b>Ticket</b><br>Issued document of carriage tied to a PNR.       |
| <b>Cancellation</b><br>Refund eligibility depends on fare rules.          | <b>No-show</b><br>Passenger did not board; impacts refund.   | <b>Remittance</b><br>Agent-to-airline payment owed for ticketed sales. | <b>Settlement</b><br>Reconciliation + payment cycle Accelya runs. |

Source: glossary and demo flows shared by Accelya. Full vocabulary handed out in the pre-read pack.

# AI & data safety — the non-negotiables

Airline data has real regulatory and contractual weight. These rules apply from Module 1 through to capstone.

## 01 Synthetic data only

No real Accelya customer or production data flows through any prompt, payload, or output. Ever.

## 02 No PII or PCI in prompts

Names, emails, card numbers, passport numbers — not in chat, not in code, not in logs.

## 03 Prompt logs are audit artefacts

/prompts/ commits to GitLab. Treat them as you would treat code review comments.

## 04 Watch for prompt injection

User-supplied content can hide instructions. Validate inputs before passing them to a model.

## 05 Cross-region data residency awareness

When APAC inference profiles are used, traffic stays within APAC destination regions. Global profiles may route worldwide — confirm the profile before sending sensitive data.

## 06 Model invocation logging is a policy decision

Bedrock can log full prompt + response to S3/CloudWatch. Trainer-managed sandbox keeps it off by default.

*If you're unsure whether a paste is safe — don't. Ask the trainer first. The cost of waiting 30 seconds is nothing.*

# Five things to take into the lab

1

**Agentic = AI proposes, human reviews, code ships.**

*Not autocomplete. Not full autonomy. The disciplined middle.*

2

**Every artefact passes the 5-stage HITL flow.**

*Prompt → Options → Review → Build → Validate — no skipping stages.*

3

**The HITL checklist has five dimensions.**

*Correctness · Security · Observability · Scalability · Cost.*

4

**Pick the right pattern: batch, streaming, or inference.**

*Latency, cost, and event shape decide — not framework preference.*

5

**Speak airline. PNR, fare class, OD, settlement.**

*Generic 'order' / 'payment' examples are banned in this course.*

*Next: guided design — we walk through the tooling stack and the HITL checklist as living artefacts.*

# 02

PART 2 · ~ 35 – 40 MINUTES

## Guided design

We walk through the tooling stack live, connect to Bedrock from Python, and see the HITL checklist as a real artefact — a printable that travels through every lab in this course.

# Approved lab stack

Five layers. Approved tooling at every level. Toolchain confirmed by Accelya: Windsurf + Cascade (the agent bundled with Windsurf — same family as Copilot Chat / Cursor Composer; should pick up .windsurfrules, plans multi-file edits, runs terminal commands, uses AWS Bedrock as model backend) + AWS Bedrock + GitLab. Cloud labs run in the client-managed AWS AI sandbox.

|   |                                                           |                                                                                                                                                                                                                                         |
|---|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | IDE & in-IDE assistant<br><b>Windsurf 2.0 · Cascade</b>   | Multi-file edits, planning, terminal awareness, MCP support, .windsurfrules per project. Cascade should pick up .windsurfrules on every prompt (Lab 1 verifies in a fresh chat); uses AWS Bedrock as the LLM backend in this programme. |
| 2 | Agentic reasoning model<br><b>AWS Bedrock</b>             | Claude / other FMs via Converse API. Architecture options, BRD drafting, prompt-driven scaffolding.                                                                                                                                     |
| 3 | Source control & CI<br><b>GitLab · Merge Requests</b>     | Private group, branch-per-learner, MR-based reviews. Lightweight CI: lint + HITL_REVIEW.md presence check.                                                                                                                              |
| 4 | Cloud platform<br><b>AWS</b>                              | S3 · Lambda · Kinesis · Glue · Athena · CloudWatch · SageMaker Serverless · IAM. Single fixed region (ap-south-1).                                                                                                                      |
| 5 | Infrastructure as code<br><b>AWS SAM · CloudFormation</b> | Deterministic deploys for every lab. No click-ops in production-shaped exercises.                                                                                                                                                       |

# Calling Bedrock from Python — Converse API

One client. One API. Consistent contract across Claude, Nova, Llama, Mistral — though some features (vision, tools) vary by model. Credentials come from IAM, not code.

verify\_bedrock.py

```
import os, boto3

client = boto3.client("bedrock-runtime", region_name="ap-south-1")

# Inference profile required – verify the approved ID in sandbox
MODEL_ID = os.getenv("BEDROCK_MODEL_ID",
    "apac.anthropic.claude-3-5-sonnet-20241022-v2:0") # APAC profile – same as D3-D12

response = client.converse(
    modelId=MODEL_ID,
    system=[{"text": "You are an airline data architect. Reply in JSON."}],
    messages=[{
        "role": "user",
        "content": [{"text": "Propose 2 architectures for an airline ticket-issue batch
pipeline."}],
    }],
    inferenceConfig={"maxTokens": 1024, "temperature": 0.2},
)

# response["usage"] → input/output token counts
print(response["output"]["message"]["content"][0]["text"])
```

## No API key

Auth uses issued IAM credentials or an assumed role via the standard boto3 credential chain.

## Inference profile, not raw ID

Raw model IDs may fail from ap-south-1 depending on model availability and access mode. Use the inference profile configured in the sandbox.

## Parameterized MODEL\_ID

Read via env var. Lets the trainer swap models per cohort without code changes.

## System prompt fenced

Tell the model its role and output shape. Reduces hallucinations and makes parsing easier.

## Token usage in response

response["usage"] gives token counts immediately. AWS/Bedrock CloudWatch metrics (Invocations · InputTokenCount · OutputTokenCount) are emitted by default — no invocation logging needed. Full prompt/response logs to S3/CW Logs require model invocation logging, which is OFF by default.

# The HITL checklist — a living artefact

Not a slide. A real file in your repository. Filled out for every AI-generated artefact you submit.

HITL\_REVIEW.md

```
module: 1
artefact: ticket-issue-validator scaffold
prompt: /prompts/m1-validator-v2.md
reviewer: <your-gitlab-handle>
```

|                                     |                      |                                                                          |
|-------------------------------------|----------------------|--------------------------------------------------------------------------|
| <input checked="" type="checkbox"/> | <b>Correctness</b>   | Real APIs only. Edge cases for malformed PNR + missing fare_class.       |
| <input checked="" type="checkbox"/> | <b>Security</b>      | IAM least-privilege. No public buckets. Secrets via env, not code.       |
| <input checked="" type="checkbox"/> | <b>Observability</b> | Structured JSON logs with correlation_id. Errors classified.             |
| <input type="checkbox"/>            | <b>Scalability</b>   | <i>Idempotent on replay. Stateless handler. Safe for 10× batch size.</i> |
| <input checked="" type="checkbox"/> | <b>Cost</b>          | Memory + timeout right-sized. No unintended Bedrock loops.               |

→ Open MR blocked: Scalability dimension unmet. Author retries with idempotency note.

## How it travels with you

- 1

**Generated**  
Every AI artefact ships with a pre-filled HITL\_REVIEW.md skeleton next to the code.
- 2

**Filled in**  
The reviewer ticks each dimension. An empty box = unmet, not skipped.
- 3

**Committed**  
The filled checklist commits to GitLab alongside the code in the same branch.
- 4

**MR-reviewed**  
Reviewer reads the code and the HITL\_REVIEW.md together — not the code in isolation.
- 5

**Auditable**  
End of program: 12 modules × N artefacts = a complete audit trail per learner.

# 03

PART 3 · ~ 2 – 2 ¼ HOURS

## Hands-on labs

Three labs, in order. Lab 1 verifies the stack. Lab 2 turns a one-line airline business requirement into a structured technical prompt. Lab 3 calls Bedrock for two architecture options and runs a HITL review on the winner.

# Starter repo — what lives where

Every lab and every artefact has a fixed home. This layout stays constant across all 12 modules.

accelya-agentic-aws/

```

|— briefs/           # 1-line BRs per module
|— docs/            # templates, guides
|— glossary/        # airline vocabulary
|— prompts/         # YOUR prompts (audit trail)
|— outputs/         # raw model responses
|— reviews/        # scorecards (CSV)
|— src/             # code (Module 5+)
|— scripts/         # verify, cleanup
|— HITL_REVIEW.md   # the checklist
|— .windsurfrules   # Cascade context
|— README.md

```

## STATIC — trainer-provided

briefs/, docs/, glossary/, src/skeletons — read-only reference. Don't edit.

## GREEN — you commit here every module

prompts/, outputs/, reviews/ — the audit trail. Every artefact has a prompt, an output, a scorecard.

## YELLOW — your decisions live here

HITL\_REVIEW.md and .windsurfrules — update per module. These travel with your code.

## BRANCH NAMING

module-<n>-<your-name>. Day 1 branch: module-1-<your-name>. Day 2 keeps the same branch open.

# Environment verification

Confirm every layer of the stack works from your laptop before we touch synthetic airline lab data.

**TIME****30 min****DIFFICULTY****Foundation****MODE****Solo + pair check****DELIVERABLE****Green run on /verify****OBJECTIVE**

By the end of this lab, you will have:

- Cloned the course starter repo from GitLab into Windsurf
- Verified Cascade is signed in and responding in your project
- Confirmed AWS sandbox access in ap-south-1 (Mumbai)
- Listed available Bedrock inference profiles from the CLI
- Successfully run `verify_bedrock.py` end-to-end

**PREREQUISITES — BRING WITH YOU**

- **Windsurf 2.0+** installed; signed in to your Windsurf account.
- **AWS CLI v2** configured with the IAM credentials trainer issued.
- **Python 3.11+** plus boto3 (latest) installed locally.
- **GitLab access** to the accelya-agentic-aws group.
- **Bedrock model access** granted in ap-south-1 (handled by trainer).

# Four steps. Twenty minutes if everything works.

1

## Clone & open in Windsurf

```
git clone git@gitlab.com:accelya-agentic-aws/starter.git && cd starter && windsurf .
```

### EXPECTED

Cascade panel opens. Repo tree visible. .windsurfrules should be picked up automatically (Lab 1 verifies in a fresh chat).

2

## Test Cascade with a non-AWS prompt

```
@cascade "Read README.md and tell me the lab structure."
```

### EXPECTED

Cascade reads the file, returns a structured summary. Cost shown in tokens.

3

## Verify AWS access in Mumbai

```
aws sts get-caller-identity --region ap-south-1
aws bedrock list-inference-profiles --region ap-south-1 \
  --query 'inferenceProfileSummaries[?contains(inferenceProfileId,
`claude`)].inferenceProfileId'
```

### EXPECTED

Your IAM ARN prints. Profile list includes the approved APAC Claude profile (apac.anthropic.claude-\*); a global.\* profile only appears if the trainer explicitly approves an override.

4

## Run the verify script

```
python scripts/verify_bedrock.py # trainer-provided in the starter repo
```

### EXPECTED

Two architecture options print as JSON. No traceback. Tokens used  $\leq$  1500.

# Success criteria & common gotchas



## YOU ARE READY FOR LAB 2 IF ...

- **verify\_bedrock.py** exits 0 and prints two architecture options as parseable JSON.
- **Cascade** can edit a file in your starter repo and the diff renders cleanly in Windsurf.
- **GitLab MR creation** works from your branch — even if the MR is empty.
- **AWS sandbox calls** succeed in ap-south-1 with the IAM user issued to you.
- **response["usage"]** is non-zero — inputTokens and outputTokens both > 0 in the Converse response.



## TOP FIVE GOTCHAS — CHECK FIRST

### Wrong region

Default region not ap-south-1 → boto3 hits us-east-1 and fails. Set `AWS_DEFAULT_REGION`.

### No Bedrock model access

`AccessDeniedException` on converse. Trainer needs to grant model access from the Bedrock console.

### Old boto3

`ClientError: Unknown operation 'Converse'`. Run `pip install -U boto3 (≥ 1.34.119, latest preferred)`.

### Wrong model identifier

Raw model IDs may fail from ap-south-1. Use the approved APAC inference profile (apac.anthropic.claude-\*); a global.\* profile only if the trainer explicitly approves an override.

### GitLab SSH key missing

`git clone` fails with permission denied. Add SSH key to your GitLab profile first.

# Business requirement → technical prompt

Take a one-line airline business ask. Turn it into a structured prompt your Bedrock model can act on.

|        |            |       |                        |
|--------|------------|-------|------------------------|
| TIME   | DIFFICULTY | MODE  | DELIVERABLE            |
| 45 min | All bands  | Pairs | /prompts/m1-arch-v1.md |

## YOUR ONE-LINE BUSINESS REQUIREMENT

*“We receive a daily ticket-issue file from each airline; we need to validate it, store it for analytics, and flag bad records for ops.”*

## Why this needs to become a technical prompt

|                            |                                                                |                             |                                                                         |
|----------------------------|----------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------|
| <b>Ambiguity</b>           | “Daily file” — CSV? JSON? Schema? File size? Arrival window?   | <b>Implicit assumptions</b> | “Validate” — against what rules? Schema only, or referential integrity? |
| <b>Missing constraints</b> | Where to store? S3 zones? Retention? Encryption? Cost ceiling? | <b>No success criteria</b>  | What does “flag” mean operationally? Email? Queue? Dashboard?           |

# Anatomy of a technical prompt

Five sections. The order matters. The discipline of writing them is half the work.

|    |                                                                               |                                                                                                                         |
|----|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 01 | <b>Role</b><br><i>Tell the model what kind of expert it is.</i>               | "You are a senior AWS data architect specialising in airline batch pipelines."                                          |
| 02 | <b>Context</b><br><i>Business setting + constraints + airline vocabulary.</i> | "Accelya processes daily ticket-issue files. Required fields: PNR, airline_code, fare_class, OD, settlement_date."      |
| 03 | <b>Task</b><br><i>What to produce. Be unambiguous.</i>                        | "Propose two architectures for ingesting and validating these files. Compare them on cost, latency, and reviewability." |
| 04 | <b>Non-goals</b><br><i>What NOT to include. Prevents over-engineering.</i>    | "Do not propose Glue Streaming, Step Functions, or any GCP/Azure service. No multi-region designs."                     |
| 05 | <b>Output format</b><br><i>Specify exact structure for parsing + review.</i>  | "Respond as JSON with keys: option_name, services, data_flow, trade_offs, estimated_monthly_cost."                      |

# From BR to prompt — the actual file you commit

BR (1 line)

*“Daily ticket-issue file → validate, store, flag bad records.”*



## Authoring tip

Ask Cascade to scaffold the five sections. Then refine with airline vocabulary by hand.

```
@cascade
Read /docs/prompt-template.md
and scaffold a 5-section prompt
for the BR in /briefs/m1.md.
Use airline vocabulary from
/glossary/airline.md.
```

/prompts/m1-arch-v1.md

### ## Role

You are a senior AWS data architect specialising in airline batch pipelines.

### ## Context

Accelya processes daily ticket-issue files (CSV, ~50MB) per airline. Required fields: PNR, airline\_code, fare\_class, OD, settlement\_date. Files arrive at S3 raw zone via SFTP gateway.

### ## Task

Propose two architectures for ingesting and validating these files. Compare on cost, latency, and reviewability.

### ## Non-goals

Do not propose Glue Streaming, Step Functions, or any GCP/Azure service. No multi-region designs.

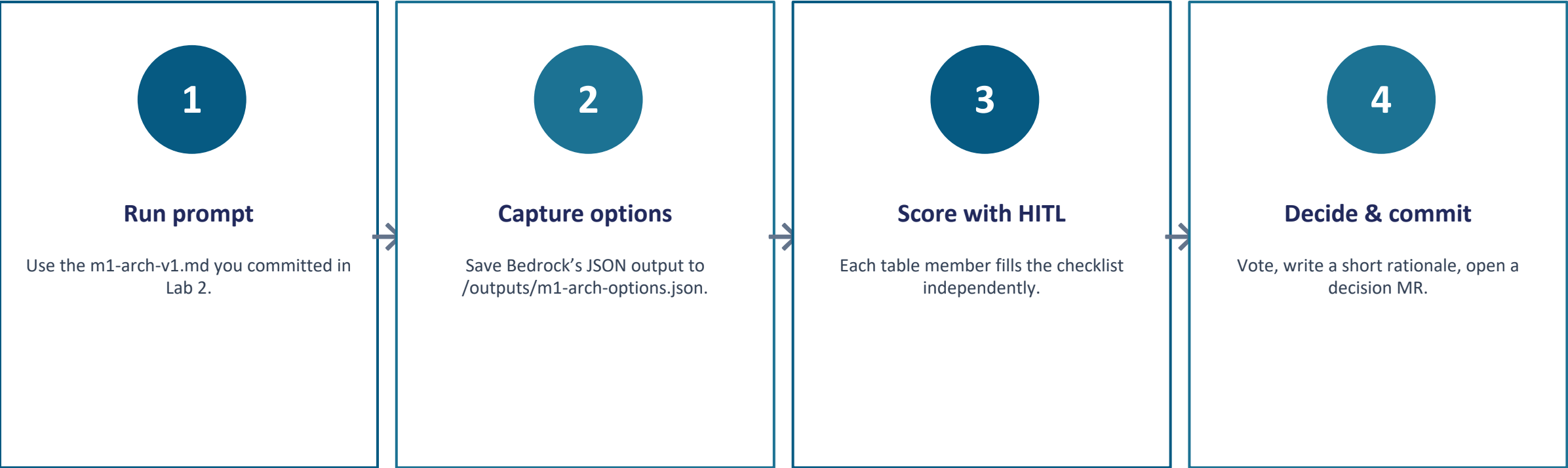
### ## Output format

Respond as JSON: {option\_name, services, data\_flow, trade\_offs, estimated\_monthly\_cost}

# Two architectures — generate, score, choose

Bedrock returns options. You apply the HITL checklist. The team picks the winner with reasons committed to GitLab.

| TIME   | DIFFICULTY | MODE                | DELIVERABLE             |
|--------|------------|---------------------|-------------------------|
| 60 min | Working+   | Table groups of 3–4 | scorecard + decision MR |



*Trainer reviews each table's decision MR live. Common pitfall: skipping the rationale — reasons matter more than the verdict.*

# Sample architecture comparison

OPTION A · S3 → Lambda → Athena

**Services:** S3 (raw, validated, curated zones) · Lambda · Glue Catalog · Athena · CloudWatch.

**Flow:** S3 PUT → Lambda validates → validated/ or quarantine/ → second Lambda transforms to Parquet in curated/ → Athena queries via Catalog.

**Trade-offs:** Cheap, simple. Lambda 15-min ceiling for very large files. Re-run = re-upload.

**Est. monthly cost:** ~\$45 (illustrative - 10 airlines, 50MB/day, low query volume).

HITL SCORE (0-3 per dim · PASS ≥12/15 AND every dim ≥2)

|      |     |     |      |      |
|------|-----|-----|------|------|
| Corr | Sec | Obs | Scal | Cost |
| 3    | 3   | 2   | 2    | 3    |

OPTION B · S3 → Glue Job → Glue Catalog

**Services:** S3 · Glue ETL job (PySpark) · Glue Data Catalog · Athena · CloudWatch.

**Flow:** EventBridge schedule → Glue job validates + transforms → partitioned Parquet in curated/.

**Trade-offs:** Scales further. Slower cold start. Higher fixed cost. PySpark = steeper review.

**Est. monthly cost:** ~\$160 (illustrative - DPU-hours dominate at this volume).

HITL SCORE (0-3 per dim · PASS ≥12/15 AND every dim ≥2)

|      |     |     |      |      |
|------|-----|-----|------|------|
| Corr | Sec | Obs | Scal | Cost |
| 3    | 3   | 3   | 3    | 1    |

Scoring rule (programme-wide from Day 1): each dimension 0-3 (0=missing · 1=weak · 2=acceptable · 3=strong). Option A = 3+3+2+2+3 = 13/15 · all dims ≥2 → PASS. Option B = 3+3+3+3+1 = 13/15 BUT Cost = 1 (<2) → REVISE before merge.

Costs are illustrative — actual pricing depends on Lambda memory/duration, Athena scan volume, Glue DPU count, S3 requests, and CW ingest.

Day 1 • Module 1 • Program Foundation & the Agentic Engineering Model

Jai Singh 30

# Decision rationale & success criteria

The score is the input. The decision is yours. Write the reasoning down — reviewers in Module 12 will read it.

MR #14 • decision: m1-architecture

**Decision:** Option A (S3 → Lambda → Athena).

**Why we chose A:**

- At our file size (50MB/day/airline), Lambda’s 15-min ceiling is comfortable.
- Lower cost matters more than scale headroom we won’t use.
- Easier review path — plain Python over PySpark for the working band.

**What we will revisit:**

- If file size grows past ~200MB, we revisit Glue (Module 6).
- Observability score (3) — will be addressed in Module 5 with structured logs + CloudWatch dashboards.

**HITL evidence:** /outputs/m1-arch-options.json, /reviews/m1-scorecard.csv

✓ YOUR LAB 3 IS COMPLETE WHEN

- **Two architecture options** saved as parseable JSON in /outputs/.
- **Each option scored on five HITL dimensions** by every table member — not just the loudest one.
- **Scorecard committed** to GitLab as a CSV with reviewer initials per row.
- **Decision MR opened** with rationale, what-we-will-revisit notes, and links to evidence.
- **HITL\_REVIEW.md filled** on the chosen option — unmet dimensions documented honestly.

# What you just built — in one frame

**LAB 1**

## Stack ready

Windsurf + Cascade + GitLab + AWS Bedrock all green

**ARTEFACT**

verify\_bedrock.py exits 0

**LAB 2**

## Prompt ready

BR transformed into a structured 5-section prompt

**ARTEFACT**

/prompts/m1-arch-v1.md committed

**LAB 3**

## Decision shipped

Two architectures generated, scored, chosen with reasons

**ARTEFACT**

Decision MR opened on GitLab

*Next: review, Q&A, cleanup, and a preview of Day 2 — AWS fundamentals on the same sandbox.*

# 04

PART 4 · ~ 20 – 30 MINUTES

## Review & wrap

We close Day 1 by naming the HITL mistakes worth avoiding, running cleanup, walking through Day 2, and — most importantly — hearing your questions and sticking points.

# Five HITL mistakes — and what to do instead

These all came up in Lab 3 reviews. Naming them now means you won't repeat them in Module 2 onwards.

**01****DON'T**

Judging the output without reading the prompt

**INSTEAD**

The prompt is the contract. Read it before you score the architecture.

**02****DON'T**

Ticking checkboxes without naming evidence

**INSTEAD**

Each tick gets a one-line reason. "Correctness ✓" alone is meaningless.

**03****DON'T**

Silent rewrite of AI output instead of feedback

**INSTEAD**

Refine the prompt. Re-run. Commit both versions. Show the iteration.

**04****DON'T**

Choosing the option that "feels right" over the scored winner

**INSTEAD**

Either trust the scorecard, or change the scorecard — don't override silently.

**05****DON'T**

Forgetting to commit the prompt log alongside code

**INSTEAD**

/prompts/<artefact>.md travels with every artefact. No prompt = not reviewable.

# End-of-module cleanup

Resources you created today must be torn down. The client-managed AWS AI sandbox (Pluralsight-provisioned) enforces daily resource caps automatically; trainer monitors invocation totals.

```
scripts/cleanup_module1.sh    # Linux/WSL example – macOS/Windows: use
verify_cleanup.sh

# 1. Empty your learner prefix in S3 (programme bucket – same as D3-D12)
aws s3 rm s3://accelya-airline-data-ap-south-1/learner-sandbox/$LEARNER/ \
  --recursive --region ap-south-1

# 2. Confirm no runaway scripts / repeated invocation loops (cross-platform
UTC):
START=$(python3 -c 'from datetime import datetime,timedelta,timezone;
print((datetime.now(timezone.utc)-
timedelta(hours=1)).isoformat().replace("+00:00","Z"))')
END=$(python3 -c 'from datetime import datetime,timezone;
print(datetime.now(timezone.utc).isoformat().replace("+00:00","Z"))')
aws cloudwatch get-metric-statistics \
  --namespace AWS/Bedrock --metric-name Invocations \
  --start-time $START --end-time $END --period 300 --statistics Sum --
region ap-south-1

# 3. Stage outputs in your branch
git add prompts/ outputs/ reviews/ HITL_REVIEW.md
git commit -m "module-1: complete"

# 4. Push branch & open MR on GitLab
git push origin module-1-$LEARNER

# 5. Verify cleanup
./verify_cleanup.sh    # expects: ALL CLEAR
```

## Cleanup checklist

- ☐ s3://accelya-airline-data-ap-south-1/learner-sandbox/\$LEARNER/ prefix is empty (verified by `aws s3 ls`)
- ☐ Invocation count within expected range — no runaway script left running
- ☐ All prompts, outputs, reviews committed to your branch
- ☐ HITL\_REVIEW.md filled and committed
- ☐ GitLab branch pushed; MR opened
- ☐ verify\_cleanup.sh prints ALL CLEAR

### ⚠️ COST GUARDRAIL

Sandbox caps fire per learner (enforced by Pluralsight) — Bedrock 50 inv / 20K tok, Lambda 10 functions, SageMaker 1 endpoint, Glue 3 jobs / 4 DPU / 6 DPU-h. Untorn-down Kinesis streams (Modules 7+) and orphaned SageMaker endpoints (Module 9) are the two recurring culprits.

# Day 1 in numbers

A snapshot of what every learner walks out of Module 1 with. This is the baseline for the next 11 modules.

5

stages

of HITL flow practiced

2

options

scored against the checklist

1

MR

decision committed to GitLab

12

terms

of airline vocabulary introduced and reused

ARTEFACTS COMMITTED TO YOUR GITLAB BRANCH TODAY

/prompts/m1-arch-v1.md

/outputs/m1-arch-options.json

/reviews/m1-scorecard.csv

HITL\_REVIEW.md (chosen option)

Decision MR rationale

verify\_bedrock.py (working baseline)

# Day 2 preview — AWS fundamentals for the workshop

Tomorrow we move from agentic concepts into the AWS services this program actually uses. Same sandbox, same starter repo.

## DAY 2 STRUCTURE · 4 HOURS

30–45 min

### Concepts

IAM · S3 · Lambda · CloudWatch · event-driven design

45–60 min

### Guided design

S3 zone model: raw → validated → curated. Permissions boundaries.

2–2¼ hrs

### Labs

Sign in · inspect S3 layout · deploy starter Lambda · read CloudWatch logs

20–30 min

### Review & wrap

Cold-start signature · IAM least-privilege · cleanup

## HOW DAY 2 BUILDS ON TODAY

### Day 1's outputs become Day 2's inputs

#### Your verified Bedrock setup

→ Remains verified — Module 2 sets up the AWS pieces around it (S3, Lambda, IAM).

#### The HITL checklist

→ Gets applied to a real Lambda — IAM least-privilege, logs, error handling.

#### Your `/prompts/m1-arch-v1.md`

→ Becomes the basis for Module 2's S3 zone design prompt.

#### Your GitLab branch

→ Stays open. Module 2's work commits to the same starter repo.

#### Airline vocabulary

→ Same terms reappear in payloads, IAM tags, log fields.

# Pre-reads & prep for Day 2

Twenty minutes tonight saves fifty minutes of catch-up tomorrow morning. The Foundation band especially benefits.

## MANDATORY · ~20 MINUTES

### 1 AWS IAM basics

Users, roles, policies, permissions boundaries. Just the docs landing page is enough.

### 2 Amazon S3 essentials

Buckets, prefixes, event notifications, versioning, lifecycle rules — conceptual only.

### 3 AWS Lambda fundamentals

Runtime, handler signature, timeout, memory, IAM role, cold/warm start.

### 4 Amazon CloudWatch

Log groups, log streams, metrics, basic alarms. Skip Insights for now.

### 5 Python `lambda_handler` refresher

`def handler(event, context):` — reading JSON, accessing nested keys, structured logs.

## ENVIRONMENT · CONFIRM TONIGHT

### ✓ Windsurf still signed in

Open it now — don't discover SSO timeouts at 09:30.

### ✓ AWS CLI default region

ap-south-1. Run `aws configure` if it isn't.

### ✓ `boto3` ≥ 1.34.119

`pip install -U boto3` if your Module 1 verify call needed it. Latest preferred.

### ✓ GitLab SSH key on this laptop

`git ls-remote` should succeed silently against the starter repo.

### ✓ `verify_bedrock.py` still passes

Run it once tonight to confirm tokens flow.

END OF DAY 1 · MODULE 1 OF 12

# You shipped the foundation.

*HITL, airline vocabulary, Windsurf + Cascade + GitLab + Bedrock + AWS — wired together. Day 2 builds on every line of it.*

---

Delivered by **Jai Singh** · *Day 2 starts on the same sandbox. See you there.*